Power Query Exercise 2

Transforming Data with the Northwind Dataset

Data Analytics | Power BI Desktop | Intermediate Level

Duration	Level	Tool	Dataset
45-60 min	**Intermediate**	**Power BI Desktop**	**Northwind**

1. Overview

In this exercise you will use Power Query in Power BI Desktop to import, clean, and transform multiple tables from the Northwind sample database. You will practice combining queries, creating calculated columns, and shaping data for analysis.

Learning Objectives

By the end of this exercise you will be able to:

- Connect Power BI to CSV source files and load multiple related tables
- Apply data type corrections and rename columns for clarity
- Filter rows and remove unnecessary columns
- Merge (join) queries to combine data from related tables
- Append queries to stack rows from similar tables
- Create custom columns using M formula expressions
- Group and aggregate data using Group By
- Control which queries load into the data model

The Northwind Dataset

Northwind is a fictional food import/export company. You will work with the six tables listed below.

Table	Key Columns	Rows (approx.)
Orders	OrderID, CustomerID, EmployeeID, OrderDate, ShippedDate, Freight	830
Order Details	OrderID, ProductID, UnitPrice, Quantity, Discount	2,155
Products	ProductID, ProductName, CategoryID, UnitPrice, UnitsInStock	77

Customers	CustomerID, CompanyName, City, Country	91
Employees	EmployeeID, FirstName, LastName, Title, Region	9
Categories	CategoryID, CategoryName, Description	8

2. Setup

Step 1 — Download the Northwind Dataset

The Northwind dataset is included with this exercise as a ZIP file named Northwind_Dataset.zip, located in the same folder as this document.

Included Files in Northwind_Dataset.zip

File	Rows	Key Columns
Categories.csv	8	CategoryID, CategoryName
Customers.csv	91	CustomerID, CompanyName, Country
Employees.csv	9	EmployeeID, FirstName, LastName, Title
Products.csv	77	ProductID, ProductName, CategoryID, UnitPrice, UnitsInStock, ReorderLevel
Orders.csv	830	OrderID, CustomerID, EmployeeID, OrderDate, Freight
Order_Details.csv	~2,470	OrderID, ProductID, UnitPrice, Quantity, Discount

Extract the ZIP to a folder on your computer, for example:

```
C:\Users\YourName\Documents\Northwind\
```

Important

Keep all 6 CSV files in the same folder — Power Query stores the path and expects all files there.

Do not rename the files — the steps below assume the exact file names shown above.

Step 2 — Open Power BI Desktop

1. Launch Power BI Desktop.
2. On the Home ribbon select Get data > Text/CSV.
3. Navigate to your Northwind folder, select Orders.csv, then click Transform Data to open Power Query Editor.
4. Repeat for: Order_Details.csv, Products.csv, Customers.csv, Employees.csv, and Categories.csv.

After loading all six files, six queries will appear in the Queries pane on the left.

Checkpoint
Six queries should appear in the Queries pane:
Orders | Order_Details | Products | Customers | Employees | Categories

3. Part A — Clean and Shape Each Table

Clean each table individually before combining them.

Task A1 — Clean the Orders Table

1. Select the Orders query in the Queries pane.
2. Promote headers: if the first row contains column names click Use First Row as Headers in the Transform ribbon.
3. Set data types: OrderDate and ShippedDate → Date; Freight → Decimal Number; OrderID, EmployeeID → Whole Number; CustomerID → Text.
4. Rename columns: ShipName → Ship Name, ShipAddress → Ship Address, ShipCity → Ship City, ShipCountry → Ship Country.
5. Remove columns: right-click ShipRegion and ShipPostalCode and choose Remove Column.
6. Filter rows: click the dropdown on OrderDate and uncheck null.

Solution — Applied Types (Orders)

```
// M code generated in Applied Steps after data type changes:
= Table.TransformColumnTypes(#"Promoted Headers",{
    {"OrderID", Int64.Type},
    {"CustomerID", type text},
    {"EmployeeID", Int64.Type},
    {"OrderDate", type date},
    {"ShippedDate", type date},
    {"Freight", type number}
})
```

Expected Result

Orders table: ~830 rows, no nulls in OrderDate, columns clearly named.
Applied Steps pane shows: Source, Promoted Headers, Changed Type, Removed Columns, Filtered Rows.

Task A2 — Clean the Order Details Table

1. Set data types: Quantity → Whole Number; UnitPrice and Discount → Decimal Number.
2. Add a custom column named Line Total using Add Column > Custom Column.
3. Set Line Total data type to Decimal Number.
4. Rename the query to OrderDetails (no space).

Solution — Line Total Column (OrderDetails)

```
// Custom Column formula:
= [UnitPrice] * [Quantity] * (1 - [Discount])

// M step generated:
= Table.AddColumn(#"Changed Type", "Line Total",
    each [UnitPrice] * [Quantity] * (1 - [Discount]),
    type number)
```

Expected Result

OrderDetails: 2,155 rows with a new Line Total column.
Sample row: UnitPrice=14.00, Quantity=12, Discount=0 → Line Total = 168.00
Sample row: UnitPrice=9.80, Quantity=10, Discount=0.15 → Line Total = 83.30

Task A3 — Clean the Products Table

1. Set data types: UnitPrice → Decimal Number; UnitsInStock and ReorderLevel → Whole Number.
2. Add a conditional column named Stock Status using Add Column > Conditional Column.
3. Remove the Discontinued column if all values are false/0.

Solution — Stock Status Column (Products)

```
// Conditional Column configuration:
//   If  UnitsInStock = 0              then "Out of Stock"
//   Else If UnitsInStock <= ReorderLevel then "Low Stock"
//   Else                                   "In Stock"

// M step generated:
= Table.AddColumn(#"Changed Type", "Stock Status", each
```

```
    if [UnitsInStock] = 0 then "Out of Stock"
    else if [UnitsInStock] <= [ReorderLevel] then "Low Stock"
    else "In Stock",
    type text)
```

Expected Result

Products: 77 rows with a new Stock Status column.
Distribution: In Stock (majority), Low Stock (a few), Out of Stock (any with 0 units).
Example: Chai — UnitsInStock=39, ReorderLevel=10 → In Stock

Task A4 — Quick Cleans: Customers, Employees, Categories

Table	Action	How
Customers	Set CustomerID to Text. Trim whitespace from CompanyName.	Transform > Format > Trim
Employees	Add Full Name column. Set HireDate to Date.	Add Column > Custom Column
Categories	Confirm CategoryID is Whole Number.	Transform > Data Type

Solution — Full Name Column (Employees)

```
// Full Name column formula (Employees):
= [FirstName] & " " & [LastName]

// M step generated:
= Table.AddColumn(#"Changed Type", "Full Name",
    each [FirstName] & " " & [LastName],
    type text)
```

Expected Result

Employees: 9 rows, new Full Name column.
Examples: Nancy Davolio, Andrew Fuller, Janet Leverling, Margaret Peacock

4. Part B — Merge Queries (Joins)

Merge Queries combines columns from two tables based on a matching key — similar to a SQL JOIN.

Join Type Reference

Left Outer — All rows from the left table; matching rows from the right (most common)
Inner — Only rows that match in both tables
Full Outer — All rows from both tables (nulls where no match)
Left Anti — Rows in the left with NO match in the right (find orphan records)

Task B1 — Add Product Name to Order Details

1. Select the OrderDetails query.
2. In the Home ribbon click Merge Queries > Merge Queries as New.
3. Set left table to OrderDetails and right table to Products. Click ProductID in both tables.
4. Set Join Kind to Left Outer. Click OK.
5. Click the expand icon on the new Products column. Select only ProductName and CategoryID. Uncheck Use original column name as prefix.
6. Rename the new query SalesDetail.

Solution — Merge OrderDetails + Products (SalesDetail)

```
// M code for the merge step:
= Table.NestedJoin(
    OrderDetails,
    {"ProductID"},
    Products,
    {"ProductID"},
    "Products",
    JoinKind.LeftOuter
)

// Expand step:
= Table.ExpandTableColumn(
    #"Merged Queries",
    "Products",
    {"ProductName", "CategoryID"},
    {"ProductName", "CategoryID"}
)
```

Expected Result

SalesDetail: 2,155 rows with ProductName and CategoryID added.

All OrderDetails rows are preserved (Left Outer — no rows lost).
Example row: OrderID=10248, ProductID=11, ProductName="Queso Cabrales", Qty=12, Line Total=168.00

Task B2 — Enrich Orders with Customer and Employee Data

1. Select the Orders query.
2. Merge with Customers on CustomerID (Left Outer). Expand CompanyName and Country. Rename to Customer Name and Customer Country.
3. Merge the same query with Employees on EmployeeID (Left Outer). Expand Full Name. Rename it Sales Rep.
4. Rename this enriched query EnrichedOrders.

Solution — EnrichedOrders

```
// Merge with Customers:
= Table.NestedJoin(Orders, {"CustomerID"}, Customers, {"CustomerID"}, "Customers",
JoinKind.LeftOuter)

// Expand CompanyName and Country:
= Table.ExpandTableColumn(#"Merged Queries", "Customers",
    {"CompanyName", "Country"}, {"Customer Name", "Customer Country"})

// Merge with Employees:
= Table.NestedJoin(#"Expanded Customers", {"EmployeeID"}, Employees,
{"EmployeeID"}, "Employees", JoinKind.LeftOuter)

// Expand Full Name:
= Table.ExpandTableColumn(#"Merged Queries2", "Employees",
    {"Full Name"}, {"Sales Rep"})
```

Expected Result

EnrichedOrders: ~830 rows with Customer Name, Customer Country, and Sales Rep columns added.
Example: OrderID=10248, Customer Name="Vins et alcools Chevalier", Customer Country="France", Sales Rep="Steven Buchanan"

5. Part C — Group By and Aggregation

Group By collapses rows into aggregated results — similar to GROUP BY in SQL.

Task C1 — Total Sales by Product Category

1. Right-click SalesDetail and choose Duplicate. Rename the copy SalesByCategory.
2. In the Transform ribbon click Group By.
3. Set Group By column to CategoryID. Add aggregations: Total Revenue (Sum of Line Total) and Order Count (Count Rows).
4. Click OK.
5. Merge with Categories on CategoryID (Left Outer). Expand only CategoryName. Move it to the first column.

Solution — SalesByCategory

```
// Group By step:
= Table.Group(SalesDetail, {"CategoryID"}, {
    {"Total Revenue", each List.Sum([Line Total]), type number},
    {"Order Count",   each Table.RowCount(_), Int64.Type}
})

// Merge + expand CategoryName:
= Table.NestedJoin(#"Grouped Rows", {"CategoryID"}, Categories, {"CategoryID"},
"Categories", JoinKind.LeftOuter)
= Table.ExpandTableColumn(#"Merged Queries", "Categories", {"CategoryName"},
{"CategoryName"})
```

Expected Result

SalesByCategory: 8 rows (one per category).
Expected output (approximate, sorted descending by revenue):

Beverages — Total Revenue: $267,869 | Order Count: 404
Dairy Products — Total Revenue: $234,507 | Order Count: 367
Confections — Total Revenue: $167,358 | Order Count: 334
Meat/Poultry — Total Revenue: $163,023 | Order Count: 173
(remaining 4 categories below $100,000)

Task C2 — Monthly Order Volume

1. Duplicate EnrichedOrders and rename the copy MonthlyOrders.
2. Add a custom column Order Month.
3. Group By Order Month. Add Order Count (Count Rows) and Total Freight (Sum of Freight).

4. Sort ascending by Order Month.

Solution — MonthlyOrders

```
// Order Month custom column:
= Date.ToText([OrderDate], "yyyy-MM")

// Group By step:
= Table.Group(#"Added Custom", {"Order Month"}, {
    {"Order Count",   each Table.RowCount(_), Int64.Type},
    {"Total Freight", each List.Sum([Freight]), type number}
})

// Sort step:
= Table.Sort(#"Grouped Rows", {{"Order Month", Order.Ascending}})
```

Expected Result

MonthlyOrders: 23 rows (July 1996 through May 1998).
Sample rows:
1996-07 — Order Count: 22, Total Freight: $1,288.41
1996-08 — Order Count: 25, Total Freight: $1,117.93
1997-01 — Order Count: 55, Total Freight: $3,969.74 (peak month)

6. Part D — Append Queries

Append stacks rows from two or more queries that share the same column structure — similar to SQL UNION ALL.

Task D1 — Combine UK and USA Customers

1. Right-click Customers and choose Reference. Rename the new query Customers_UK.
2. Filter the Country column to show only United Kingdom.
3. Right-click Customers again and choose Reference. Rename it Customers_USA. Filter Country to USA.
4. In the Home ribbon click Append Queries > Append Queries as New.
5. Select Two tables: Customers_UK (first) and Customers_USA (second). Click OK.
6. Rename the result Customers_UK_USA.
7. Verify: row count of Customers_UK_USA = Customers_UK rows + Customers_USA rows.

Solution — Append Customers_UK + Customers_USA

```
// Customers_UK filter step:
= Table.SelectRows(Customers, each [Country] = "United Kingdom")

// Customers_USA filter step:
= Table.SelectRows(Customers, each [Country] = "USA")

// Append step (Customers_UK_USA):
= Table.Combine({Customers_UK, Customers_USA})
```

Expected Result

Customers_UK: 7 rows (UK-based customers)
Customers_USA: 13 rows (US-based customers)
Customers_UK_USA: 20 rows (7 + 13 combined)
All columns remain identical — CompanyName, City, Country, etc.

Reference vs. Duplicate

Reference — new query sources from the original; upstream changes propagate automatically.
Duplicate — an independent copy; upstream changes do NOT affect it.
Best practice: use Reference for filtered views of the same source.

7. Part E — Advanced Column Techniques

Task E1 — Extract Year, Quarter, and Fiscal Period

Work in the EnrichedOrders query.

1. Use Add Column > Date > Year to add Order Year.
2. Use Add Column > Date > Quarter of Year to add Order Quarter.
3. Add a custom text column Fiscal Period combining both.

Solution — Date Columns (EnrichedOrders)

```
// Order Year (via ribbon – generates):
= Table.AddColumn(#"...", "Order Year",
    each Date.Year([OrderDate]), Int64.Type)

// Order Quarter (via ribbon – generates):
= Table.AddColumn(#"...", "Order Quarter",
```

```
    each Date.QuarterOfYear([OrderDate]), Int64.Type)

// Fiscal Period custom column formula:
= "Q" & Text.From(Date.QuarterOfYear([OrderDate]))
  & " " & Text.From(Date.Year([OrderDate]))

// M step generated:
= Table.AddColumn(#"...", "Fiscal Period",
    each "Q" & Text.From(Date.QuarterOfYear([OrderDate]))
         & " " & Text.From(Date.Year([OrderDate])),
    type text)
```

Expected Result

Three new columns added to EnrichedOrders.

Sample row (OrderDate = 1996-07-04): Order Year = 1996 | Order Quarter = 3 | Fiscal Period = "Q3 1996"

Sample row (OrderDate = 1997-01-15): Order Year = 1997 | Order Quarter = 1 | Fiscal Period = "Q1 1997"

Task E2 — Categorise Orders by Freight Cost

Add a conditional column Freight Band to EnrichedOrders:

Condition	Output Value
Freight < 25	Low
Freight >= 25 and < 100	Medium
Freight >= 100	High

Solution — Freight Band Column

```
// Conditional Column configuration:
//   If  Freight < 25              then "Low"
//   Else If Freight < 100         then "Medium"
//   Else                          "High"

// M step generated:
= Table.AddColumn(#"...", "Freight Band",
    each if [Freight] < 25 then "Low"
         else if [Freight] < 100 then "Medium"
         else "High",
    type text)
```

Expected Result

Freight Band column added to EnrichedOrders (~830 rows).
Approximate distribution: Low ~35%, Medium ~45%, High ~20%.
Example: Freight=32.38 → Medium | Freight=3.17 → Low | Freight=148.33 → High

8. Loading Queries to the Data Model

Staging queries support the final queries but do not need to appear in the report model.

1. Right-click Customers_UK and uncheck Enable Load. Repeat for Customers_USA.
2. Queries to keep enabled: EnrichedOrders, OrderDetails, SalesDetail, SalesByCategory, MonthlyOrders, Products, Customers, Categories, Employees, Customers_UK_USA.
3. Click Close & Apply on the Home ribbon.
4. Switch to Model view and verify all relationships exist, or create them manually.

Expected Relationships

Orders (OrderID) 1 : * Order_Details (OrderID)
Products (ProductID) 1 : * Order_Details (ProductID)
Customers (CustomerID) 1 : * Orders (CustomerID)
Employees (EmployeeID) 1 : * Orders (EmployeeID)
Categories (CategoryID) 1 : * Products (CategoryID)

Expected Result

Power Query Editor closes and data loads into the model.
Model view shows 5 relationships (auto-detected or manually created).
Customers_UK and Customers_USA appear greyed out (load disabled) in the Queries pane.

9. Challenge Extension

Completed all tasks early? Try the extension activities below.

A	Build a SalesByCountry query: total revenue and order count per customer country. Sort descending by revenue.
B	Add a Rank column to SalesByCategory: sort descending by Total Revenue, then add an Index column starting at 1 (this index acts as the rank).

C	Create a Date parameter StartDate. Filter EnrichedOrders to return only orders on or after that date. Test with at least two different values.
D	Open the Advanced Editor on any one query. Explain in plain English what each M step does and why Power Query generated it.

10. Reflection Questions

Answer the following questions based on your work in this exercise.

1. What is the difference between Merge Queries and Append Queries? Give a real-world scenario for each.

---

2. In Task B1 you used a Left Outer join. What rows would be lost if you had used an Inner join instead?

---

3. Why is Reference preferred over Duplicate when creating filtered subsets of a query?

---

4. What does the Enable Load toggle control? Why might you want to disable load on a staging query?

---

5. In Task C1 you grouped by CategoryID rather than CategoryName. Why might this be the safer choice?

---

6. Describe a situation where a Left Anti join would be useful in the Northwind data.

---

11. Solution Section

- This section provides the complete M code and expected outputs for every task in the exercise.
- Review this section after you have attempted the work independently.

Part A — Complete M Code

Task A1 — Orders (complete Applied Steps)

```
let
    Source              =
Csv.Document(File.Contents("C:\\Northwind\\Orders.csv"),[Delimiter=",",
Encoding=1252]),
    PromotedHeaders     = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
    ChangedType         = Table.TransformColumnTypes(PromotedHeaders,{
                              {"OrderID", Int64.Type}, {"CustomerID", type text},
                              {"EmployeeID", Int64.Type}, {"OrderDate", type date},
                              {"ShippedDate", type date}, {"Freight", type
number}}),
    RenamedCols         = Table.RenameColumns(ChangedType,{
                              {"ShipName","Ship Name"},{"ShipAddress","Ship
Address"},
                              {"ShipCity","Ship City"},{"ShipCountry","Ship
Country"}}),
    RemovedCols         = Table.RemoveColumns(RenamedCols,
{"ShipRegion","ShipPostalCode"}),
    FilteredRows        = Table.SelectRows(RemovedCols, each [OrderDate] <> null)
in
    FilteredRows
```

Task A2 — OrderDetails (complete Applied Steps)

```
let
    Source          =
Csv.Document(File.Contents("C:\\Northwind\\Order_Details.csv"),[Delimiter=",",
Encoding=1252]),
    PromotedHeaders = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
    ChangedType     = Table.TransformColumnTypes(PromotedHeaders,{
                          {"OrderID", Int64.Type}, {"ProductID", Int64.Type},
                          {"UnitPrice", type number}, {"Quantity", Int64.Type},
                          {"Discount", type number}}),
    AddedLineTotal  = Table.AddColumn(ChangedType, "Line Total",
                          each [UnitPrice] * [Quantity] * (1 - [Discount]), type
number)
in
    AddedLineTotal
```

Task A3 — Products (complete Applied Steps)

```
let
    Source           =
Csv.Document(File.Contents("C:\\Northwind\\Products.csv"),[Delimiter=",",
Encoding=1252]),
    PromotedHeaders  = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
    ChangedType      = Table.TransformColumnTypes(PromotedHeaders,{
                           {"ProductID", Int64.Type}, {"CategoryID", Int64.Type},
                           {"UnitPrice", type number}, {"UnitsInStock",
Int64.Type},
                           {"ReorderLevel", Int64.Type}}),
    AddedStockStatus = Table.AddColumn(ChangedType, "Stock Status",
                           each if [UnitsInStock] = 0 then "Out of Stock"
                                else if [UnitsInStock] <= [ReorderLevel] then "Low
Stock"
                                else "In Stock", type text),
    RemovedDiscont   = Table.RemoveColumns(AddedStockStatus, {"Discontinued"})
in
    RemovedDiscont
```

Task A4 — Employees Full Name

```
let
    Source          =
Csv.Document(File.Contents("C:\\Northwind\\Employees.csv"),[Delimiter=",",
Encoding=1252]),
    PromotedHeaders = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
    ChangedType     = Table.TransformColumnTypes(PromotedHeaders,{
                          {"EmployeeID", Int64.Type}, {"HireDate", type date}}),
    AddedFullName   = Table.AddColumn(ChangedType, "Full Name",
                          each [FirstName] & " " & [LastName], type text)
in
    AddedFullName
```

Part B — Complete M Code

Task B1 — SalesDetail

```
let
    MergedQueries  = Table.NestedJoin(OrderDetails, {"ProductID"}, Products,
{"ProductID"},
                         "Products", JoinKind.LeftOuter),
    ExpandedCols   = Table.ExpandTableColumn(MergedQueries, "Products",
                         {"ProductName", "CategoryID"}, {"ProductName",
"CategoryID"})
in
    ExpandedCols
```

T

ask B2 — EnrichedOrders

```
let
    MergeCustomers   = Table.NestedJoin(Orders, {"CustomerID"}, Customers,
{"CustomerID"},
                           "Customers", JoinKind.LeftOuter),
    ExpandCustomers  = Table.ExpandTableColumn(MergeCustomers, "Customers",
                           {"CompanyName", "Country"}, {"Customer Name", "Customer
Country"}),
    MergeEmployees   = Table.NestedJoin(ExpandCustomers, {"EmployeeID"},
Employees, {"EmployeeID"},
                           "Employees", JoinKind.LeftOuter),
    ExpandEmployees  = Table.ExpandTableColumn(MergeEmployees, "Employees",
                           {"Full Name"}, {"Sales Rep"})
in
    ExpandEmployees
```

Part C — Complete M Code

Task C1 — SalesByCategory

```
let
    GroupedRows     = Table.Group(SalesDetail, {"CategoryID"}, {
                          {"Total Revenue", each List.Sum([Line Total]), type
number},
                          {"Order Count",   each Table.RowCount(_), Int64.Type}}),
    MergeCategories = Table.NestedJoin(GroupedRows, {"CategoryID"}, Categories,
                          {"CategoryID"}, "Categories", JoinKind.LeftOuter),
    ExpandName      = Table.ExpandTableColumn(MergeCategories, "Categories",
                          {"CategoryName"}, {"CategoryName"}),
    ReorderedCols   = Table.ReorderColumns(ExpandName,
                          {"CategoryName", "CategoryID", "Total Revenue", "Order
Count"})
in
    ReorderedCols
```

Task C2 — MonthlyOrders

```
let
    AddedMonth   = Table.AddColumn(EnrichedOrders, "Order Month",
                       each Date.ToText([OrderDate], "yyyy-MM"), type text),
    GroupedRows  = Table.Group(AddedMonth, {"Order Month"}, {
                       {"Order Count",   each Table.RowCount(_), Int64.Type},
                       {"Total Freight", each List.Sum([Freight]), type number}}),
    SortedRows   = Table.Sort(GroupedRows, {{"Order Month", Order.Ascending}})
in
    SortedRows
```

Part D — Complete M Code

Task D1 — Customers_UK_USA

```
// Customers_UK query:
let
    FilteredUK = Table.SelectRows(Customers, each [Country] = "United Kingdom")
in FilteredUK

// Customers_USA query:
let
    FilteredUSA = Table.SelectRows(Customers, each [Country] = "USA")
in FilteredUSA

// Customers_UK_USA query:
let
    AppendedRows = Table.Combine({Customers_UK, Customers_USA})
in AppendedRows
```

Part E — Complete M Code

Task E1 — Date Columns

```
let
    // Starting from EnrichedOrders ...
    AddedYear     = Table.AddColumn(EnrichedOrders, "Order Year",
                         each Date.Year([OrderDate]), Int64.Type),
    AddedQuarter  = Table.AddColumn(AddedYear, "Order Quarter",
                         each Date.QuarterOfYear([OrderDate]), Int64.Type),
    AddedFiscal   = Table.AddColumn(AddedQuarter, "Fiscal Period",
                         each "Q" & Text.From(Date.QuarterOfYear([OrderDate]))
                              & " " & Text.From(Date.Year([OrderDate])), type text)
in
    AddedFiscal
```

Task E2 — Freight Band

```
= Table.AddColumn(#"Added Fiscal Period", "Freight Band",
    each if [Freight] < 25 then "Low"
         else if [Freight] < 100 then "Medium"
         else "High",
    type text)
```

Challenge Extension — Solutions

Challenge A — SalesByCountry

```
let
    GroupCountry   = Table.Group(EnrichedOrders, {"Customer Country"}, {
                         {"Total Revenue", each List.Sum([Freight]), type number},
                         {"Order Count",   each Table.RowCount(_), Int64.Type}}),
    // Note: join SalesDetail to get true revenue if needed
    SortedDesc     = Table.Sort(GroupCountry, {{"Total Revenue",
Order.Descending}})
in
    SortedDesc
```

Challenge B — Rank Column on SalesByCategory

```
let
    SortedDesc  = Table.Sort(SalesByCategory, {{"Total Revenue",
Order.Descending}}),
    AddedIndex  = Table.AddIndexColumn(SortedDesc, "Rank", 1, 1, Int64.Type)
in
    AddedIndex
```

Challenge C — StartDate Parameter

```
// Create parameter (Manage Parameters > New Parameter):
//   Name: StartDate
//   Type: Date
//   Current Value: 1997-01-01  (example)

// Filter step in EnrichedOrders:
= Table.SelectRows(#"Added Freight Band",
    each [OrderDate] >= StartDate)
```

Reflection Question — Suggested Answers

1. Merge vs. Append

Merge (Join) combines COLUMNS from two tables based on a matching key — e.g. joining OrderDetails to Products to pull in ProductName. Append stacks ROWS from tables with the same structure — e.g. combining Customers_UK and Customers_USA into one list.

2. Left Outer vs. Inner join in Task B1

A Left Outer join keeps all 2,155 rows from OrderDetails whether or not a matching Product exists. An Inner join would drop any OrderDetails rows whose ProductID does not exist in Products — potentially losing order line data for discontinued or missing products.

3. Reference vs. Duplicate

A Reference query sources from the original; if the original is updated (e.g. a new data type step is added), the Reference inherits the change automatically. A Duplicate is an independent copy that does not propagate upstream changes, leading to maintenance effort and potential inconsistency.

4. Enable Load

Enable Load controls whether a query's results are pushed into the Power BI data model (and appear as a table in Report/Data view). Disabling it on staging queries (e.g. Customers_UK) keeps the model lean — fewer tables means faster refresh and a cleaner field list for report authors.

5. Grouping by CategoryID vs. CategoryName

CategoryID is the primary key and guaranteed unique. CategoryName could theoretically have duplicates or minor spelling differences ("Beverages" vs "beverages") that would create false split groups. Grouping by the ID first, then joining CategoryName in afterwards, is more robust.

6. Left Anti join use case

A Left Anti join on Orders vs. OrderDetails (joining on OrderID) would return any Orders that have NO line items — i.e. empty or corrupt orders. Similarly, a Left Anti join on Products vs. OrderDetails would show products that have never been ordered, useful for inventory analysis.